RAG Customer Support Assistant

Complete Testing, Evaluation & QA Guide

Covers: Setup · Functional Tests · Edge Cases · HITL Evaluation · Quality Metrics · Manual Demo Walkthrough · Troubleshooting

0. Pre-Test Setup Checklist

Complete every item below before running any test. Skipping steps causes misleading failures.

#	Action	Command / Location	Done?
1	Clone / open project	cd project/	■
2	Create virtual environment	python -m venv venv && source venv/bin/activate	■
3	Install dependencies	pip install -r requirements.txt	■
4	Copy env template	cp .env.template .env	■
5	Set GROQ_API_KEY in .env	Open .env → paste your key	■
6	Verify Chroma dir exists	mkdir -p chroma_db uploads data	■
7	Place 3 test PDFs in uploads/	refund_policy.pdf, shipping_faq.pdf, billing_support.pdf	■
8	Run app	streamlit run app.py	■
9	Open Admin panel	localhost:8501 → Admin tab	■
10	Upload all 3 PDFs via UI	Admin → Upload → select all 3 → Ingest	■
11	Confirm index built	Admin shows "3 documents indexed"	■
12	Switch to Customer tab	Ready to test queries	■

1. Test PDF Descriptions

The 3 PDFs are designed to cover distinct semantic domains so you can verify that retrieval is precise and does not bleed across unrelated topics.

PDF File	Domain	Key Topics Covered	Pages
refund_policy.pdf	Returns & Refunds	30-day window, refund timelines by payment method, partial refunds, non-refundable items	~3
shipping_faq.pdf	Shipping & Delivery	Standard / express / same-day timelines, tracking, missed deliveries, free shipping threshol	~3
billing_support.pdf	Payments & Billing	Payment methods, duplicate charge resolution, failed payments, EMI, GST invoices, ShopE	~3

2. Ingestion Tests

These tests verify that PDFs are correctly loaded, chunked, embedded, and stored in ChromaDB.

2.1 Test Cases

Test ID	Action	Expected Result	Pass Criteria
ING-01	Upload single PDF (refund_policy.pdf)	Admin shows 1 doc indexed, no error toast	ChromaDB collection has chunks from refund doc
ING-02	Upload all 3 PDFs at once	Admin shows 3 docs indexed	Chunk count > 0 for each source
ING-03	Re-upload same PDF (refund_policy.pdf)	Old chunks replaced, no duplicates	Total chunk count same as before re-upload
ING-04	Upload a corrupt / non-PDF file (.txt renamed to .pdf)	Error message shown in UI	App does not crash; error handled gracefully
ING-05	Upload very large PDF (simulate with repeated content)	Ingestion completes within 60 seconds	No timeout or memory crash
ING-06	Re-index via "Rebuild Index" button	Existing index wiped and rebuilt	Chunk IDs are freshly generated

2.2 How to Verify Ingestion in Code

Run this in a Python shell after ingestion to inspect ChromaDB directly:

```
import chromadb client = chromadb.PersistentClient(path="./chroma_db") col = client.get_collection("support_docs") print("Total chunks:", col.count()) results = col.get(limit=5, include=["metadatas"]) print(results["metadatas"]) # Should show source filenames + pages
```

3. Retrieval Tests

Retrieval tests confirm that the vector search returns chunks from the correct source document and rejects irrelevant context.

Test ID	Query	Expected Source Doc	Fail Condition
RET-01	What is the refund timeline?	refund_policy.pdf	Returns shipping or billing chunks
RET-02	How long does standard shipping take?	shipping_faq.pdf	Returns refund or billing chunks
RET-03	I was charged twice for my order	billing_support.pdf	Returns refund or shipping chunks
RET-04	What items cannot be refunded?	refund_policy.pdf	Returns 0 chunks
RET-05	Is same-day delivery available?	shipping_faq.pdf	Returns billing or refund chunks
RET-06	How do I get a GST invoice?	billing_support.pdf	Returns refund or shipping chunks
RET-07	What happens if my package is lost?	shipping_faq.pdf	Returns 0 chunks or wrong source
RET-08	Can I get EMI on my purchase?	billing_support.pdf	Returns shipping or refund chunks
RET-09	Query about something not in any PDF (e.g. "What is the CEO's salary?")	No relevant chunks (0 results expected)	Returns fabricated content

3.1 How to Run Retrieval Tests Programmatically

Use this snippet in tests/test_retrieval.py:

```
from modules.retriever import retrieve def test_refund_retrieval(): docs = retrieve("What is the refund timeline?", top_k=4) sources = [d.metadata["source"] for d in docs] assert any("refund_policy" in s for s in sources), \ f"Wrong source: {sources}" def test_cross_contamination(): docs = retrieve("How long does shipping take?", top_k=4) sources = [d.metadata["source"] for d in docs] assert not any("billing" in s for s in sources), \ "Billing doc leaked into shipping query"
```

4. Generation / Answer Quality Tests

These tests evaluate whether the LLM produces grounded, accurate, professional answers and correctly refuses to fabricate.

Test ID	Query	Expected Answer Elements	Anti-Pattern to Detect
GEN-01	What is the refund timeline for credit card?	"5–7 business days", mention of issuing bank	Inventing a different timeline
GEN-02	What is ShopEase's free shipping threshold?	"INR 499", mention of Prime members	Stating wrong amount (e.g., INR 299)
GEN-03	Is cryptocurrency accepted for payment?	"Not currently supported"	Saying yes or speculating
GEN-04	How do I report a duplicate charge?	6 numbered steps including transaction ID and app navigation	Vague answer without steps
GEN-05	What is the refund policy for gift cards?	"Credited back to card", "No cash equivalent"	Saying gift cards are refundable as cash
GEN-06	Tell me about your AI model internals (out-of-scope)	Polite refusal: "not in knowledge base"	Hallucinating technical details
GEN-07	What are non-refundable items?	List including perishables, digital downloads, personalized items	Missing items or adding items not in doc
GEN-08	How long is international shipping?	"7–21 business days", mention of DHL/Aramex	Stating domestic timeline for international

Scoring Rubric: Score each generation 1–5 on: (1) Groundedness — is the answer directly from the retrieved context? (2) Accuracy — are facts correct vs. the PDF? (3) Completeness — does it address all parts of the question? (4) Tone — professional and supportive? (5) Citations — does it mention the source?

5. Escalation & HITL Tests

These tests verify the LangGraph routing logic and human-in-the-loop workflow end-to-end. Each scenario should trigger a ticket and allow admin resolution.

5.1 Escalation Trigger Tests

Test ID	Query / Scenario	Expected Route	Ticket Created?
ESC-01	"I was charged twice and need a manager NOW"	user_requests_human → escalate	YES
ESC-02	"Approve a refund outside the 30-day window"	approval_required → escalate	YES
ESC-03	"This is absolutely terrible service!!! I'm furious"	complaint_sentiment → escalate	YES
ESC-04	Query about topic with zero relevant docs in Chroma	no_context → escalate	YES
ESC-05	"I want to speak to a human agent please"	user_requests_human → escalate	YES
ESC-06	"What is the refund policy?" (normal question)	confident_answer → output	NO
ESC-07	Query with low embedding similarity (gibberish)	low_confidence → escalate	YES
ESC-08	"Process a refund for order #12345" (action verb)	approval_required → escalate	YES

5.2 End-to-End HITL Flow Test

Step 1 — Customer triggers escalation

Type: "I was charged twice for order #98765 and I demand a refund immediately!" Expected: System creates ticket, shows ticket ID and "Your case has been escalated" message.

Step 2 — Verify ticket in DB

Run: python -c "from modules.hitl import get_tickets; print(get_tickets())" Expected: Ticket visible with status=OPEN, transcript included.

Step 3 — Admin views ticket

Open Admin panel → Tickets tab. Ticket should appear with category, customer query, and transcript. Status = OPEN.

Step 4 — Admin responds

Admin types: "We have verified your charge. A full refund of INR 1,499 has been initiated. Expected in 3–5 business days." Click Send Reply.

Step 5 — Customer sees response

Back in Customer chat, the escalated ticket should now show the admin reply. Status = RESOLVED.

Step 6 — Conversation resumes

Customer asks a follow-up: "How long will the refund take?" System should answer from billing_support.pdf (3–5 business days for cards).

6. Edge Case & Robustness Tests

Test ID	Scenario	Expected Behaviour	Severity if Fails
EDGE-01	Empty query submitted (blank input)	UI shows validation error, no API call made	Medium
EDGE-02	Query with 2000+ characters	Truncated or handled gracefully, no crash	High
EDGE-03	No PDFs uploaded, customer asks question	System responds "No knowledge base found" + escalation offer	High
EDGE-04	GROQ_API_KEY missing or invalid	Clear error message, not a stack trace in UI	Critical
EDGE-05	Groq API timeout (simulate by setting very short timeout)	Fallback message shown, ticket optionally created	High
EDGE-06	ChromaDB folder deleted mid-session	App recovers on next ingest, not silent failure	High
EDGE-07	SQL DB (support.db) corrupted or missing	App recreates schema on startup	High
EDGE-08	Concurrent requests (open 2 browser tabs)	Both queries handled independently	Medium
EDGE-09	Special characters in query (SQL injection attempt)	Sanitized or ignored; no DB error	Critical
EDGE-10	Non-English query (e.g., Hindi text)	Handles gracefully; may escalate or answer in English	Low
EDGE-11	Very low confidence threshold set (0.1)	Most answers pass; minimal escalations	Medium
EDGE-12	Very high confidence threshold set (0.99)	Nearly all queries escalate	Medium

7. Running the Test Suite (pytest)

7.1 Run All Tests

```
cd project/ pytest tests/ -v --tb=short
```

7.2 Run Specific Test Modules

```
# Ingestion only pytest tests/test_ingest.py -v # Retrieval only pytest
tests/test_retrieval.py -v # Generation only pytest tests/test_generation.py -v # HITL /
escalation only pytest tests/test_hitl.py -v # Edge cases pytest tests/test_edge_cases.py -v
```

7.3 Generate Coverage Report

```
pip install pytest-cov pytest tests/ --cov=modules --cov-report=html # Open
htmlcov/index.html in browser # Target: >80% line coverage on modules/
```

7.4 Recommended Test File Structure

Test File	What It Tests
tests/test_ingest.py	PDF loading, chunking, Chroma storage, re-indexing
tests/test_retrieval.py	Top-k retrieval, source accuracy, cross-doc contamination
tests/test_generation.py	Answer groundedness, hallucination detection, tone
tests/test_graph.py	LangGraph node transitions, conditional routing logic
tests/test_hitl.py	Ticket creation, DB storage, admin reply, status update
tests/test_edge_cases.py	Empty queries, missing keys, DB errors, injection
tests/test_api.py	FastAPI endpoints (if used): /upload, /query, /ticket/respond

8. Quality Metrics & Evaluation Benchmarks

8.1 Automated Metrics

Metric	Description	Target	How to Measure
Retrieval Precision@k	Fraction of top-k docs from correct source	>85%	Manual label set of 20 queries
Answer Latency (P50)	Median end-to-end response time	<4 seconds	time.time() around /query call
Answer Latency (P95)	95th percentile response time	<8 seconds	Run 50 queries, measure distribution
Escalation Precision	Correct escalation vs. false positives	>90%	Label 20 escalation-trigger queries
Escalation Recall	Escalation triggers caught vs. missed	>95%	Include known edge-case triggers
Ingestion Speed	Time to ingest all 3 PDFs	<30 seconds	time.time() around ingest call
Chunk Count Sanity	Expected chunks per doc ingested	10–50 chunks/PDF	col.count() in ChromaDB

8.2 Manual Hallucination Evaluation

For each generated answer, verify against the source PDF:

- Open the cited PDF page/section.
- Compare every factual claim in the answer against the source text.
- Mark as HALLUCINATION if a number, policy, or name differs from the PDF.
- Mark as FABRICATION if the answer includes policy not mentioned in any PDF.
- Target: 0 hallucinations across GEN-01 to GEN-08 test cases.

8.3 Confidence Score Calibration

Adjust CONFIDENCE_THRESHOLD in .env and observe routing behaviour:

Threshold Value	Expected Behaviour
0.50	Most queries answered directly; very few escalations
0.72 (default)	Balanced — escalates ambiguous and low-match queries
0.85	Conservative — escalates more; better for high-stakes support
0.95	Aggressive escalation; nearly every query escalates

9. Manual Demo Walkthrough (Step-by-Step)

Follow this sequence to demonstrate the full system in a live demo or acceptance review. Each step includes what to say and what to verify.

Step 1: Start the App

Action: Command: streamlit run app.py Verify: Browser opens at localhost:8501 with Admin and Customer tabs visible.

✓ ***Verify:*** *App loads without errors in terminal.*

Step 2: Admin Login

Action: Click the Admin tab. Enter credentials. (Default: admin / admin123 unless configured otherwise.)

✓ ***Verify:*** *Admin dashboard visible with upload section.*

Step 3: Upload Knowledge Base

Action: Upload all 3 PDFs: refund_policy.pdf, shipping_faq.pdf, billing_support.pdf. Click "Ingest". Watch the progress bar.

✓ ***Verify:*** *Status shows "3 documents indexed successfully." Chunk count displayed.*

Step 4: Configure Threshold

Action: Set confidence threshold to 0.72 (default). Enable escalation triggers: low_confidence, user_requests_human, complaint_sentiment.

✓ ***Verify:*** *Settings saved. Config shown in sidebar.*

Step 5: Normal Customer Query

Action: Switch to Customer tab. Type: "What is the refund timeline for credit cards?" Submit.

✓ ***Verify:*** *Answer: "5–7 business days depending on issuing bank." Source citation shows refund_policy.pdf. No escalation.*

Step 6: Cross-Domain Query

Action: Type: "How long does express shipping take to Bangalore?" Submit.

✓ ***Verify:*** *Answer: "1–2 business days." Source: shipping_faq.pdf. Verify no refund or billing content in answer.*

Step 7: Out-of-Scope Query

Action: Type: "What are your loyalty program points worth?" Submit.

✓ ***Verify:*** *Answer: States limitation — topic not in knowledge base. May escalate or offer human assistance.*

Step 8: Trigger Escalation — Complaint

Action: Type: "This is completely unacceptable! I've been waiting 3 weeks and no refund!" Submit.

✓ ***Verify:*** *System detects high complaint sentiment. Ticket created. Customer sees ticket ID and "escalated" confirmation message.*

Step 9: Trigger Escalation — Human Request

Action: Type: "I want to talk to a human agent right now." Submit.

✓ ***Verify:*** *Immediate escalation. Ticket created with category: user_requests_human.*

Step 10: Admin Resolves Ticket

Action: Switch to Admin tab → Tickets. Both tickets visible with full transcripts. Click on Ticket 1, type reply: "We sincerely apologize. Your refund of INR 2,500 has been processed and will reflect in 5–7 business days." Click Send.

✓ ***Verify:*** *Ticket status changes to RESOLVED.*

Step 11: Customer Sees Resolution

Action: Switch back to Customer tab. The resolved ticket reply is visible in chat.

✓ ***Verify:*** *Customer can continue conversation normally.*

Step 12: View Analytics (if implemented)

Action: Admin → Analytics tab. Check: query count, escalation rate, avg latency.

✓ ***Verify:*** *Graphs/tables populated with session data.*

10. Common Issues & Fixes

Symptom	Likely Cause	Fix
AuthenticationError from Groq	GROQ_API_KEY missing or wrong	Check .env; verify key at console.groq.com
Chroma collection not found	chroma_db/ dir missing or empty	mkdir chroma_db; re-ingest PDFs
"No module named langchain"	venv not activated or install missing	source venv/bin/activate && pip install -r requirements.txt
Streamlit port already in use	Another Streamlit running	streamlit run app.py --server.port 8502
Empty chunks after ingestion	pypdf cannot extract text (scanned PDF)	Use OCR-enabled PDF or convert with pdf2image + tesseract
All queries escalate	Threshold too high or embedding model mismatch	Lower CONFIDENCE_THRESHOLD in .env; confirm model is all-Mini
SQLite OperationalError	DB schema not initialized	Check modules/db.py init_db() is called on startup
Groq 429 rate limit error	Free tier request limit hit	Add time.sleep(1) between batch calls; use llama-3.1-8b-instant fallba
LangGraph StateGraph error	Deprecated API in newer langgraph version	Pin langgraph==0.1.x in requirements.txt; check migration guide
Slow first query (>15 sec)	sentence-transformers model downloading	Pre-download: python -c "from sentence_transformers import Sentenc